

Getting Started with Videos

Goal

- Learn to read video, display video and save video.
- Learn to capture from Camera and display it.
- You will learn these functions : **cv2.VideoCapture()**, **cv2.VideoWriter()**

Capture Video from Camera

Often, we have to capture live stream with camera. OpenCV provides a very simple interface to this. Let's capture a video from the camera (I am using the in-built webcam of my laptop), convert it into grayscale video and display it. Just a simple task to get started.

To capture a video, you need to create a **VideoCapture** object. Its argument can be either the device index or the name of a video file. Device index is just the number to specify which camera. Normally one camera will be connected (as in my case). So I simply pass 0 (or -1). You can select the second camera by passing 1 and so on. After that, you can capture frame-by-frame. But at the end, don't forget to release the capture.

```
1 import numpy as np
2 import cv2
3
4 cap = cv2.VideoCapture(0)
5
6 while(True):
7     # Capture frame-by-frame
8     ret, frame = cap.read()
9
10    # Our operations on the frame come here
11    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
12
13    # Display the resulting frame
14    cv2.imshow('frame',gray)
15    if cv2.waitKey(1) & 0xFF == ord('q'):
16        break
17
18 # When everything done, release the capture
19 cap.release()
20 cv2.destroyAllWindows()
```

cap.read() returns a bool (True/False). If frame is read correctly, it will be True. So you can check end of the video by checking this return value.

Sometimes, cap may not have initialized the capture. In that case, this code shows error. You can check whether it is initialized or not by the method **cap.isOpened()**. If it is True, OK. Otherwise open it using **cap.open()**.

You can also access some of the features of this video using **cap.get(propId)** method where propId is a number from 0 to 18. Each number denotes a property of the video (if it is applicable to that video) and full details can be seen here: [Property Identifier](#). Some of these values can be modified using **cap.set(propId, value)**. Value is the new value you want.

For example, I can check the frame width and height by cap.get(3) and cap.get(4). It gives me 640x480 by default. But I want to modify it to 320x240. Just use ret = cap.set(3,320) and ret = cap.set(4,240).

Note

If you are getting error, make sure camera is working fine using any other camera application (like Cheese in Linux).

Playing Video from file

It is same as capturing from Camera, just change camera index with video file name. Also while displaying the frame, use appropriate time for `cv2.waitKey()`. If it is too less, video will be very fast and if it is too high, video will be slow (Well, that is how you can display videos in slow motion). 25 milliseconds will be OK in normal cases.

```

1 import numpy as np
2 import cv2
3
4 cap = cv2.VideoCapture('vtest.avi')
5
6 while(cap.isOpened()):
7     ret, frame = cap.read()
8
9     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
10
11     cv2.imshow('frame',gray)
12     if cv2.waitKey(1) & 0xFF == ord('q'):
13         break
14
15 cap.release()
16 cv2.destroyAllWindows()

```

Note

Make sure proper versions of ffmpeg or gstreamer is installed. Sometimes, it is a headache to work with Video Capture mostly due to wrong installation of ffmpeg/gstreamer.

Saving a Video

So we capture a video, process it frame-by-frame and we want to save that video. For images, it is very simple, just use `cv2.imwrite()`. Here a little more work is required.

This time we create a **VideoWriter** object. We should specify the output file name (eg: output.avi). Then we should specify the **FourCC** code (details in next paragraph). Then number of frames per second (fps) and frame size should be passed. And last one is **isColor** flag. If it is True, encoder expect color frame, otherwise it works with grayscale frame.

FourCC is a 4-byte code used to specify the video codec. The list of available codes can be found in fourcc.org. It is platform dependent. Following codecs works fine for me.

- In Fedora: DIVX, XVID, MJPG, X264, WMV1, WMV2. (XVID is more preferable. MJPG results in high size video. X264 gives very small size video)
- In Windows: DIVX (More to be tested and added)
- In OSX : *(I don't have access to OSX. Can some one fill this?)*

FourCC code is passed as `cv2.VideoWriter_fourcc('M','J','P','G')` or `cv2.VideoWriter_fourcc(*'MJPG')` for MJPG.

Below code capture from a Camera, flip every frame in vertical direction and saves it.

```

1 import numpy as np
2 import cv2
3
4 cap = cv2.VideoCapture(0)
5
6 # Define the codec and create VideoWriter object
7 fourcc = cv2.VideoWriter_fourcc(*'XVID')
8 out = cv2.VideoWriter('output.avi',fourcc, 20.0, (640,480))
9
10 while(cap.isOpened()):
11     ret, frame = cap.read()
12     if ret==True:
13         frame = cv2.flip(frame,0)
14
15         # write the flipped frame
16         out.write(frame)
17
18         cv2.imshow('frame',frame)
19         if cv2.waitKey(1) & 0xFF == ord('q'):

```

```
20         break
21     else:
22         break
23
24 # Release everything if job is finished
25 cap.release()
26 out.release()
27 cv2.destroyAllWindows()
```

Additional Resources

Exercises